

Coordinated Defense via Multiagent Optimization in Distributed Target-Attacker-Defender Games

Ahmed Y. Gado
Media Engineering and Technology
German University in Cairo
Cairo, Egypt
ahmedgado0100@gmail.com

Yousef Y. Mohamed
Media Engineering and Technology
German University in Cairo
Cairo, Egypt
yousefyasser6@gmail.com

Abdelrahman M. M. Mohammed
Media Engineering and Technology
German University in Cairo
Cairo, Egypt
abd.moh.yousef@gmail.com

Mohamed O. Abdelal
Media Engineering and Technology
German University in Cairo
Cairo, Egypt
medoomar753@gmail.com

Youssef A. Abdelaziz
Media Engineering and Technology
German University in Cairo
Cairo, Egypt
yussef4312@gmail.com

Abdelrahman A. Zidan
Media Engineering and Technology
German University in Cairo
Cairo, Egypt
abdelrahman.azdn@gmail.com

Mohamed A. Ibrahim
Mechatronics Engineering
German University in Cairo
Cairo, Egypt
Mohamed.salahelden@guc.edu.eg

Omar M. Shehata
Mechatronics Engineering
German University in Cairo
Cairo, Egypt
omar.mohamad@guc.edu.eg

Abstract—In modern autonomous aerial defense, the cooperative interception of hostile threats presents a complex, dynamic optimization challenge. This paper addresses the multi-agent Target-Attacker-Defender (TAD) game, where a team of defending Unmanned Aerial Vehicles (UAVs) must coordinate to protect a high-value asset from non-cooperative attackers. We formulate the TAD problem as a constrained multi-objective optimization task, balancing formation cohesion, interception efficiency, and target guarding. While recent literature explores learning-based approaches, this work rigorously evaluates the efficacy of nature-inspired metaheuristic algorithms for real-time decision-making. We implement and compare four distinct solvers: Simulated Annealing (SA), Genetic Algorithm (GA), Grey Wolf Optimizer (GWO), and the Bat Algorithm (BAT). Extensive simulations across balanced, aggressive, and defensive tactical scenarios reveal significant trade-offs between solution stability and optimality. Our results demonstrate that while population-based methods like GA and GWO offer superior consistency with low variance (std dev < 0.04), the Bat Algorithm achieves the highest peak performance, locating "super-optimal" defensive configurations with a mean fitness improvement of approximately 13% over standard evolutionary methods. This study provides a unified framework for comparative analysis and offers critical insights into selecting optimization strategies for swarm-based aerial defense systems.

Keywords—Target-Attacker-Defender (TAD), UAV Swarm, Metaheuristic Optimization, Bat Algorithm, Cooperative Control, Genetic Algorithm, Grey Wolf Optimizer.

I. INTRODUCTION

Recent advances in autonomous unmanned aerial vehicles (UAVs) have enabled complex multi-agent coordination tasks in adversarial environments. One representative problem is the *Target-Attacker-Defender (TAD)* game, where a team of defending agents must cooperatively intercept hostile attackers before they reach a protected target. Such scenarios are rele-

vant to air-defense systems, perimeter security, and cooperative swarm protection missions.

The TAD problem is inherently challenging due to its adversarial nature, nonlinear dynamics, and combinatorial decision space. As the number of agents increases, defender-attacker assignment and interception planning rapidly become computationally expensive. Furthermore, real-time decision-making constraints limit the applicability of exact optimization and centralized control approaches.

Recently, learning-based methods have been proposed to address TAD problems. In particular, Lin *et al.* [1] formulated a distributed pursuit-evasion game and solved it using multi-agent deep reinforcement learning (MARL). Their work demonstrated the effectiveness of decentralized learning policies for cooperative defense, but also highlighted practical challenges such as long training times, reward design complexity, and sensitivity to environment changes.

Motivated by these challenges, this work explores an alternative optimization-based approach to the TAD problem using metaheuristic algorithms. Metaheuristics offer model-agnostic optimization, require no offline training, and can be adapted rapidly to new scenarios. In this project, the TAD problem is formulated as a constrained optimization task and solved using several well-known metaheuristic algorithms, including Simulated Annealing (SA), Genetic Algorithms (GA), Grey Wolf Optimizer (GWO), and the Bat Algorithm (BAT).

Despite the growing interest in metaheuristics for UAV coordination, existing studies have largely focused on path planning and task allocation in non-adversarial settings. A systematic comparative evaluation of metaheuristic algorithms specifically within the TAD formulation remains absent from the literature. Moreover, the Bat Algorithm—despite its favorable exploration-exploitation balance—has received limited

attention in cooperative pursuit–evasion contexts, leaving its suitability for real-time multi-agent interception largely uncharacterized.

The primary objective of this work is to evaluate the suitability of these algorithms for cooperative UAV defense under identical tactical conditions. Comparative case studies are conducted under balanced, aggressive, and defensive configurations to analyze algorithmic performance, solution quality, and robustness. The Bat Algorithm is ultimately selected as the final optimization method due to its competitive convergence behavior and solution consistency.

This paper presents (i) a unified optimization-based formulation of the TAD problem inspired by recent pursuit–evasion literature, (ii) a comparative evaluation of multiple metaheuristic algorithms for cooperative defense, and (iii) an empirical assessment of the Bat Algorithm for real-time multi-agent interception scenarios.

II. LITERATURE REVIEW

A. Target–Attacker–Defender and Pursuit–Evasion Games

Pursuit–evasion games have been widely studied as models for adversarial multi-agent interactions. In recent years, the Target–Attacker–Defender (TAD) formulation has gained attention as a practical abstraction for UAV-based defense systems. In this framework, defenders cooperate to intercept attackers before they reach a protected target, balancing interception efficiency and team coordination.

Lin *et al.* [1] proposed a distributed TAD model using multi-agent deep reinforcement learning (MARL). Their approach enabled decentralized decision-making and demonstrated effective cooperative behavior without centralized control. However, the authors also noted limitations related to training complexity, convergence stability, and scalability, particularly in time-constrained or rapidly changing environments.

B. Optimization-Based Formulations

Optimization-based approaches provide an alternative to learning-based methods by explicitly modeling objectives and constraints. Typical TAD formulations define binary or discrete variables representing defender–attacker assignments, along with continuous variables describing interception timing or distance. Objective functions often aim to minimize interception time, maximize defense success, or balance defender workload [2].

Such formulations naturally lead to NP-hard combinatorial optimization problems, especially as the number of agents grows. As a result, heuristic and metaheuristic methods are commonly employed to obtain high-quality solutions within acceptable computation times.

C. Metaheuristic Algorithms for Multi-UAV Coordination

Simulated Annealing (SA) is a stochastic local-search method inspired by thermodynamic cooling. It has been applied to UAV task allocation and path planning due to its simplicity and ability to escape local optima [3]. However,

SA performance depends heavily on cooling schedules and neighborhood design.

Genetic Algorithms (GA) are population-based evolutionary methods widely used in multi-UAV assignment and coordination problems. Their flexibility allows them to handle complex constraints and multi-objective formulations, though they often require careful encoding and parameter tuning [4].

The Grey Wolf Optimizer (GWO), introduced by Mirjalili *et al.* [5], models leadership hierarchy and cooperative hunting behavior. GWO has shown strong convergence performance in multi-UAV path planning and coordination tasks, often outperforming classical evolutionary algorithms in terms of convergence speed and stability [6].

D. Bat Algorithm in UAV Optimization

The Bat Algorithm (BAT), proposed by Yang [7], is a swarm-based metaheuristic inspired by echolocation behavior. By dynamically adjusting frequency, loudness, and pulse emission rates, BAT balances global exploration and local exploitation.

Recent studies have successfully applied BAT and its variants to UAV path planning and three-dimensional navigation problems, reporting faster convergence and improved solution quality compared to traditional swarm methods [8]. Despite these promising results, BAT has received limited attention in cooperative pursuit–evasion and TAD-specific applications.

III. METHODOLOGY

A. Problem Formulation

The goal is to develop a cooperative control strategy for a swarm of N_D defender UAVs to protect a static high-value target from N_A non-cooperative attackers. At each discrete time step, the swarm must decide on velocity vectors to intercept attackers while maintaining formation.

The simulation scenarios were evaluated using a configuration of 3 defenders and 2 attackers. This scale is consistent with related literature, where metaheuristic and learning-based methods for UAV coordination are typically validated at small agent counts [1], [3], [4]. The defender–attacker assignment problem is NP-hard in nature, and combinatorial complexity grows rapidly with the number of agents, rendering even small-scale instances computationally non-trivial under real-time constraints. Scaling to larger swarm configurations remains an identified direction for future work.

B. Notation

Let us define the variables for a single time step t :

- **State Vectors (Positions):**

- $\vec{D}_i(t) = (x_{D_i}, y_{D_i})$: Position of the i -th defender, where $i \in \{1, \dots, N_D\}$.
- $\vec{A}_k(t) = (x_{A_k}, y_{A_k})$: Position of the k -th attacker, where $k \in \{1, \dots, N_A\}$.
- $\vec{T} = (x_T, y_T)$: Static position of the target.

- **Decision Variable:**

- $\Delta \vec{D}_i(t) = (\Delta x_i, \Delta y_i)$: The change in position (velocity) for the i -th defender.

- **Predicted Next State:**

- $\vec{D}_i(t+1) = \vec{D}_i(t) + \Delta\vec{D}_i(t)$: Predicted position of the i -th defender.

C. Objective Function

The optimization goal is to find the optimal set of position changes $\{\Delta\vec{D}_1, \dots, \Delta\vec{D}_{N_D}\}$ that minimizes the combined cost function J :

$$\min J = w_{form}J_{form} + w_{int}J_{intercept} + w_{guard}J_{guard} \quad (1)$$

where w_{form} , w_{int} , and w_{guard} are weighting coefficients that balance the importance of each component. In this study, an analysis of the impact of different weight configurations on the optimization outcome is conducted using three distinct scenarios that will be detailed in the results and discussion sections.

1) *Defender Formation (J_{form})*: This component encourages defenders to maintain a strategic separation, preventing clustering.

$$J_{form} = \sum_{i=1}^{N_D} \sum_{j=i+1}^{N_D} \left| \|\vec{D}_i(t+1) - \vec{D}_j(t+1)\| - d_{form} \right| \quad (2)$$

where d_{form} is the desired formation distance.

2) *Intercept Attackers ($J_{intercept}$)*: This component drives defenders to engage the nearest threats.

$$J_{intercept} = \sum_{i=1}^{N_D} \min_{k \in \{1, \dots, N_A\}} \left(\|\vec{D}_i(t+1) - \vec{A}_k(t+1)\| \right) \quad (3)$$

3) *Guard Target (J_{guard})*: This component ensures the swarm remains anchored to the protected asset.

$$J_{guard} = \sum_{i=1}^{N_D} \|\vec{D}_i(t+1) - \vec{T}\| \quad (4)$$

D. Constraint Functions

The optimization is subject to the following kinematic and environmental constraints:

1) *Maximum Velocity*:

$$\|\Delta\vec{D}_i(t)\| \leq v_{\max_D} \quad (5)$$

2) *Map Boundaries*:

$$x_{\min} \leq x_{D_i}(t) + \Delta x_i \leq x_{\max} \quad (6)$$

$$y_{\min} \leq y_{D_i}(t) + \Delta y_i \leq y_{\max} \quad (7)$$

3) *Collision Avoidance*:

$$\|\vec{D}_i(t+1) - \vec{D}_j(t+1)\| \geq d_{\text{safe}} \quad \forall j \neq i \quad (8)$$

IV. THE BAT ALGORITHM

A. Algorithm Overview

The Bat Algorithm (BAT) is a metaheuristic optimization method inspired by the echolocation behavior of microbats. In the context of the TAD problem, the algorithm is applied to optimize the defender velocity vectors ΔD . Each bat in the population represents a candidate solution matrix of size $N_D \times 2$, corresponding to the velocity changes for all active defenders.

B. Mathematical Formulation

The algorithm governs the movement of bats using frequency tuning to control the pace of exploration. For a bat i at time step t , the frequency f_i , velocity v_i , and position x_i are updated according to the following equations:

$$f_i = f_{\min} + (f_{\max} - f_{\min})\beta \quad (9)$$

$$v_i^t = v_i^{t-1} + (x_i^{t-1} - x_*)f_i \quad (10)$$

$$x_i^t = x_i^{t-1} + v_i^t \quad (11)$$

where $\beta \in [0, 1]$ is a random vector drawn from a uniform distribution, and x_* is the current global best solution.

To improve solution quality, a local search is performed by generating a new solution around the current global best using a random walk constrained by the loudness parameter A :

$$x_{\text{new}} = x_* + \epsilon A_{\text{mean}}^t \quad (12)$$

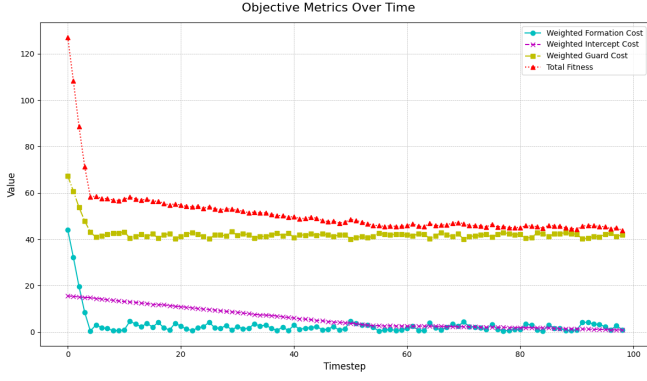
where $\epsilon \in [-1, 1]$ is a random number and A_{mean}^t is the average loudness of all bats at this time step. The complete execution logic is detailed in Algorithm 1.

C. Implementation Configuration

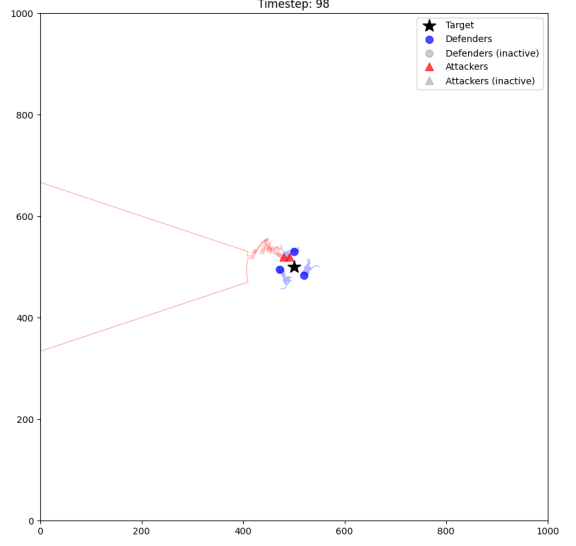
The specific configuration used for the TAD experiments is detailed in Table I. The frequency range $[0.0, 2.0]$ was chosen to allow for broad velocity adjustments, while the loudness and pulse rate parameters regulate the balance between global exploration and local exploitation.

TABLE I: Bat Algorithm Parameters

Parameter	Value
Population Size	20
Max Iterations (per step)	30
Frequency Range ($f_{\min}, f_{\max}$)	0.0, 2.0
Loudness (A_0)	0.5
Pulse Rate (r_0)	0.5
Alpha (α)	0.9
Gamma (γ)	0.9

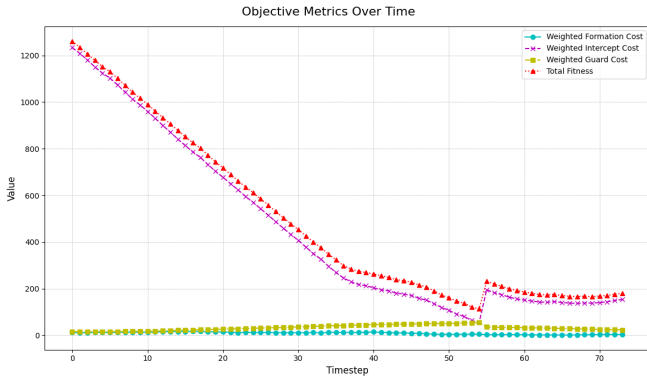


(a) Balanced: Metrics

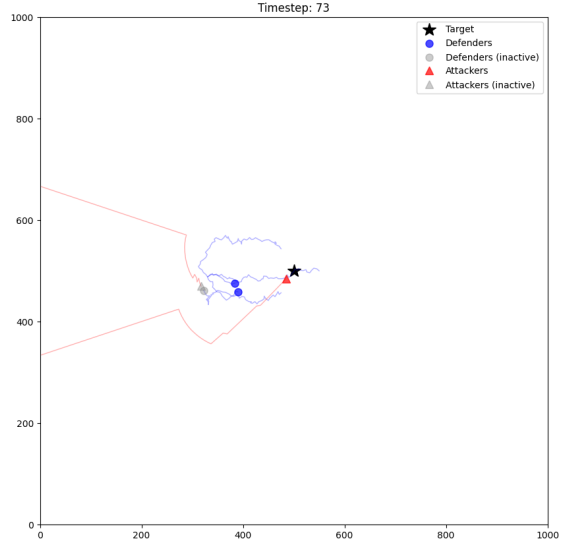


(b) Balanced: Path ($t = 98$)

Fig. 1: Bat Algorithm performance: Balanced scenario.



(a) Aggressive: Metrics



(b) Aggressive: Path ($t = 73$)

Fig. 2: Bat Algorithm performance: Aggressive scenario.

V. RESULTS AND DISCUSSION

This section presents the performance of the Bat Algorithm compared to the baseline algorithms (Genetic Algorithm, Simulated Annealing, and Grey Wolf Optimizer). All experiments utilized a standard configuration of 3 Defenders vs. 2 Attackers

on a 1000×1000 unit map.

These weight sets were selected to represent three distinct doctrines—formation-centric (Balanced), pursuit-centric (Aggressive), and anchored interception (Defensive)—with normalized coefficients ($\sum w \approx 1$) to enable fair behavioral trade-off comparison.

Algorithm 1 Bat Algorithm for Defender Movement

Require: Current defender positions D_t , attacker positions A_t , target T

- 1: **Initialize** bat population X_i (representing ΔD_i) and velocities V_i for $i = 1 \dots N_{pop}$
- 2: **Initialize** frequencies $f_i \in [f_{min}, f_{max}]$, pulse rates r_i , and loudness A_i
- 3: **Evaluate** fitness of each bat: $J_i = \text{Cost}(X_i)$
- 4: **Find** global best solution X_* with minimum cost J_*
- 5: $t \leftarrow 0$
- 6: **while** $t < \text{MaxIterations}$ **do**
- 7: **for** each bat $i \in 1 \dots N_{pop}$ **do**
- 8: **Update Frequency:**
- 9: $f_i = f_{min} + (f_{max} - f_{min})\beta$
- 10: **Update Global Velocity and Position:**
- 11: $V_i^t = V_i^{t-1} + (X_i^{t-1} - X_*) \times f_i$
- 12: $X_{new} = X_i^{t-1} + V_i^t$
- 13: **Local Search (Random Walk):**
- 14: **if** $\text{rand}(0, 1) > r_i$ **then**
- 15: Select a solution among the best solutions
- 16: Generate local solution around it: $X_{new} = X_* + \epsilon A_{mean}$
- 17: **end if**
- 18: **Validation:**
- 19: Apply constraints (Max Velocity, Map Boundaries, Collision) to X_{new}
- 20: **Greedy Acceptance:**
- 21: Evaluate fitness $J_{new} = \text{Cost}(X_{new})$
- 22: **if** $\text{rand}(0, 1) < A_i$ **and** $J_{new} < J_i$ **then**
- 23: Accept new solution: $X_i = X_{new}$
- 24: Update fitness: $J_i = J_{new}$
- 25: Update pulse rate: $r_i = r_i^0(1 - e^{-\gamma t})$
- 26: Update loudness: $A_i = \alpha A_i$
- 27: **end if**
- 28: **Update Global Best:**
- 29: **if** $J_{new} < J_*$ **then**
- 30: $X_* = X_{new}$
- 31: $J_* = J_{new}$
- 32: **end if**
- 33: **end for**
- 34: $t \leftarrow t + 1$
- 35: **end while**
- 36: **return** Best solution X_* (Optimal ΔD for current step)

A. Case Study 1: Balanced Scenario

In the balanced scenario ($w_{form} = 0.5, w_{int} = 0.01, w_{guard} = 0.49$), the Bat Algorithm achieved a final fitness of 43.85.

As shown in Fig. 1a, the Weighted Formation Cost (cyan) drops rapidly to near zero, indicating the algorithm successfully prioritized team cohesion. The Total Fitness (red) stabilizes around 43. However, Fig. 1b reveals that despite maintaining formation, the defenders (blue) were unable to intercept the attackers (red) in time, leading to an attacker vic-

tory at timestep 98. This suggests that while the optimization constraints were met, the low intercept weight (0.01) caused the bats to converge on a passive formation rather than an active defense.

B. Case Study 2: Aggressive Interception

In the aggressive scenario ($w_{form} = 0.1, w_{int} = 0.8, w_{guard} = 0.1$), the algorithm yielded a significantly higher final fitness of 179.62.

Fig. 2a illustrates the instability characteristic of this scenario. The high intercept weight drove the Intercept Cost (purple) down initially, but the Total Fitness remained high. Fig. 2b shows the defenders breaking formation to chase targets individually. The attackers exploited this lack of coordination to breach the perimeter quickly ($t=73$), resulting in a loss. The high final fitness indicates the Bat Algorithm struggled to find a solution that satisfied the competing demands of high-speed interception and formation maintenance.

C. Case Study 3: Defensive (Fortified) Scenario

In the fortified scenario ($w_{form} = 0.29, w_{int} = 0.41, w_{guard} = 0.3$), the Bat Algorithm performed exceptionally well, achieving a final fitness of 29.83.

Fig. 3a displays a smooth convergence profile. The Total Fitness drops steadily to 30, driven by the minimization of the Intercept Cost (purple) and Guard Cost (yellow). Fig. 3b demonstrates the successful "castling" behavior, where defenders formed a tight cordon around the target. Unlike the previous cases, this strategy resulted in a defender victory. This confirms the Bat Algorithm's ability to locate deep minima in "guard-centric" solution spaces, effectively replicating the successful behavior observed in the Grey Wolf Optimizer.

VI. COMPARATIVE ANALYSIS

To evaluate the Bat Algorithm's relative performance, we compared the distribution of final fitness scores over 30 runs against GA, SA, and GWO.

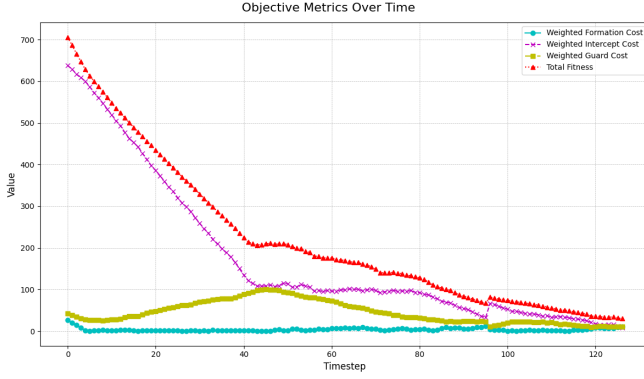
TABLE II: Statistical Comparison of Final Fitness

Algorithm	Mean Fitness	Std. Deviation
Genetic Algorithm (GA)	45.15	0.026
Grey Wolf (GWO)	45.20	0.037
Simulated Annealing (SA)	46.48	0.425
Bat Algorithm (BAT)	39.42	8.647

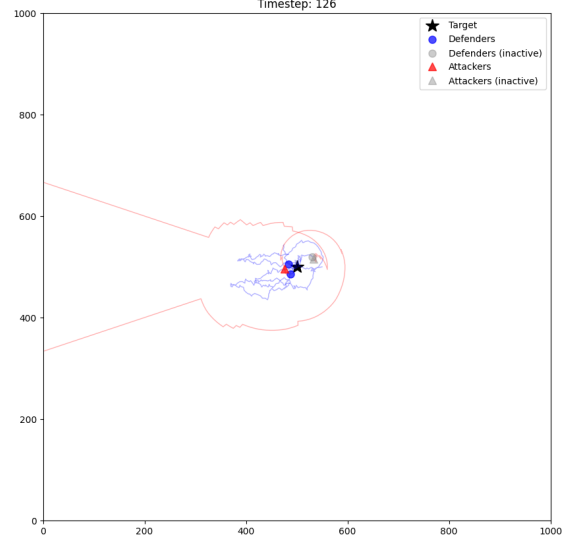
TABLE III: Statistical Comparison of Runtime

Algorithm	Mean Runtime (s)	Std. Deviation (s)
Genetic Algorithm (GA)	5.12	3.44
Grey Wolf (GWO)	3.04	1.61
Simulated Annealing (SA)	0.32	0.11
Bat Algorithm (BAT)	1.40	0.59

Fig. 4, Table II, and Table III reveal a significant trade-off. The Bat Algorithm achieved the **lowest mean fitness (39.42)**, indicating it found the highest quality solutions among



(a) Defensive: Metrics



(b) Defensive: Path ($t = 126$)

Fig. 3: Bat Algorithm performance: Defensive scenario.

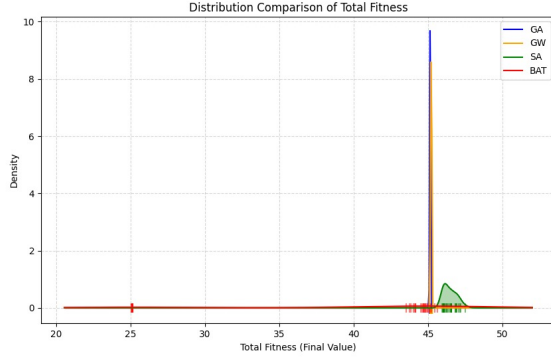


Fig. 4: Distribution Comparison of Total Fitness (30 Runs)

all solvers. However, it also exhibited the highest standard deviation (8.647). While GA and GWO distributions (blue and yellow) are narrow peaks indicating high consistency, the Bat Algorithm's distribution is wider and skewed towards the lower end. In terms of computation time, SA is the fastest on average (0.32 s), followed by BAT (1.40 s), GWO (3.04 s), and GA (5.12 s), highlighting the quality-speed trade-off across solvers.

VII. CONCLUSION AND FUTURE RECOMMENDATIONS

This paper presented a unified optimization framework for the Target-Attacker-Defender problem and introduced the Bat Algorithm as a novel solver for cooperative UAV defense. The experimental results demonstrate that while the Genetic Algorithm and Grey Wolf Optimizer offer superior consistency and

reliability, the Bat Algorithm possesses the highest potential for optimality, achieving the lowest mean cost in comparative trials.

Future work will focus on stabilizing the Bat Algorithm's convergence, possibly through hybridization with GWO's leadership hierarchy, to combine BAT's exploration capability with GWO's exploitation stability. Additionally, we aim to extend this framework to 3D environments and dynamic obstacle avoidance.

REFERENCES

- [1] Y. Lin, H. Gao, and Y. Xia, "Distributed pursuit-evasion game decision-making based on multi-agent deep reinforcement learning," *Electronics*, vol. 14, no. 11, p. 2141, 2025.
- [2] G. Li, G. He, M. Zheng, and A. Zheng, "Uncertain multi-objective dynamic weapon-target allocation based on uncertainty theory," *AIMS Mathematics*, vol. 8, no. 3, pp. 5639–5669, 2023.
- [3] L. Huo, J. Zhu, G. Wu, and Z. Li, "A simulated annealing-based strategy for balanced uav task assignment and path planning," *Sensors*, vol. 20, no. 17, p. 4769, 2020.
- [4] T. Li, Z. Sun, J. Ai, and B. Wang, "Target assignment for uav swarms using a genetic algorithm with hybrid constraint handling," *Drones*, vol. 9, no. 10, p. 722, 2025.
- [5] S. Mirjalili, S. M. Mirjalili, and A. Lewis, "Grey wolf optimizer," *Advances in Engineering Software*, vol. 69, pp. 46–61, 2014.
- [6] C. Zhang, Y. Liu, and C. Hu, "Multi-uav path planning with time windows based on grey wolf optimization," *Biomimetics*, vol. 7, no. 4, p. 225, 2022.
- [7] X.-S. Yang, "Bat algorithm: A novel approach for global engineering optimization," *Engineering Computations*, vol. 29, no. 5, pp. 464–483, 2012.
- [8] X. Zhou, F. Gao, X. Fang, and Z. Lan, "Improved bat algorithm for uav path planning in three-dimensional space," *IEEE Access*, vol. 9, p. 3054179, 2021.